SWAMI KESHVANAND INSTITUTE OF TECHNOLOGY, MANAGEMENT AND GRAMOTHAN, JAIPUR

Hands on Lab Guide
(Lab Manual)

Deep Learning LAB
IV Year B.Tech. VII SEM
(Course Code: 7CAI4-21)

Session 2025-2026

Department of Computer Science & Engineering
SKIT, JAIPUR

Swami Keshvanand Institute of Technology,

Management & Gramothan, Ramnagaria, Jagatpura, Jaipur-302017

Deep Learning Lab (7CAI4-21)

VERSION 1.0

	AUTHOR / OWNER	REVIEWED BY	APPROVED BY
NAME	Dr. Meenakshi Nawal		Prof. (Dr.) Mehul Mahrishi HOD (CSE)
DESIGNATION	Associate Professor		
SIGNATURE			
NAME			
DESIGNATION			
SIGNATURE			
NAME			
DESIGNATION			
SIGNATURE			
NAME			
DESIGNATION			
SIGNATURE			

LAB RULES

Responsibilities of Users

Users are expected to follow some fairly obvious rules of conduct:

Always:

- Enter the lab on time and leave at proper time.
- Wait for the previous class to leave before the next class enters.
- Keep the bag outside in the respective racks.
- Utilize lab hours in the corresponding.
- Turn off the machine before leaving the lab unless a member of lab staff has specifically told you not to do so.
- Leave the labs at least as nice as you found them.
- If you notice a problem with a piece of equipment (e.g. a computer doesn't respond) or the room in general (e.g. cooling, heating, lighting) please report it to lab staff immediately. Do not attempt to fix the problem yourself.

E-Mail: hodcs@skit.ac.in, URL: www.skit.ac.in

Never:

- Don't abuse the equipment.
- Do not adjust the heat or air conditioners. If you feel the temperature is not properly set, inform lab staff; we will attempt to maintain a balance that is healthy for people and machines.
- Do not attempt to reboot a computer. Report problems to lab staff.
- Do not remove or modify any software or file without permission.
- Do not remove printers and machines from the network without being explicitly told to do so by lab staff.

- Don't monopolize equipment. If you're going to be away from your machine for more than 10 or 15 minutes, log out before leaving. This is both for the security of your account, and to ensure that others are able to use the lab resources while you are not.
- Don’t use internet, internet chat of any kind in your regular lab schedule.
- Do not download or upload of MP3, JPG or MPEG files.
- No games are allowed in the lab sessions.
- No hardware including USB drives can be connected or disconnected in the labs without prior permission of the lab in-charge.
- No food or drink is allowed in the lab or near any of the equipment. Aside from the fact that it leaves a mess and attracts pests, spilling anything on a keyboard or other piece of computer equipment could cause permanent, irreparable, and costly damage. (and in fact *has*) If you need to eat or drink, take a break and do so in the canteen.
- Don’t bring any external material in the lab, except your lab record, copy and books.
- Don’t bring the mobile phones in the lab. If necessary then keep them in silence mode.
- Please be considerate of those around you, especially in terms of noise level. While labs are a natural place for conversations of all types, kindly keep the volume turned down.

Note: If you are having problems or questions, please go to either the faculty, lab in-charge or the lab supporting staff.

They will help you. We need your full support and cooperation for smooth functioning of the lab.

INSTRUCTIONS

BEFORE ENTERING IN THE LAB

- All the students are supposed to prepare the theory regarding the next experiment/ Program.
- Students are supposed to bring their lab records as per their lab schedule.
- Previous experiment/program should be written in the lab record.
- If applicable trace paper/graph paper must be pasted in lab record with proper labeling.
- All the students must follow the instructions, failing which he/she may not be allowed in the lab.

WHILE WORKING IN THE LAB

- Adhere to experimental schedule as instructed by the lab in-charge/faculty.
- Get the previously performed experiment/ program signed by the faculty/ lab in charge.
- Get the output of current experiment/program checked by the faculty/ lab in charge in the lab copy.
- Each student should work on his/her assigned computer at each turn of the lab.
- Take responsibility of valuable accessories.
- Concentrate on the assigned practical and don't play games.
- If anyone is caught red-handed carrying any equipment of the lab, then he/she will have to face serious consequences.

Marking/Assessment System

Total Marks -100

Distribution of Marks - 60 (Sessional)

Attendance	File Work	Performance	Viva	Total
10	10	20	20	60

Distribution of Marks - 40 (End Term) Depends on Examiner

File Work	Performance	Viva	Total
10	20	10	40

RAJASTHAN TECHNICAL UNIVERSITY, KOTA

Scheme & Syllabus

IV Year- VII & VIII Semester: B. Tech. Computer Science and Engineering (AI)

7CAI4-21: Deep Learning and its Applications Lab

Credit: 2 **Max. Marks: 100 (IA:60, ETE:40)**
0L+0T+4P **End Term Exam: 2 Hours**

SN	List of Experiments
1	Build a deep neural network model start with linear regression using a) Single variable b) Multiple variables
2	Write a program to convert : a) Speech into text b) Text into speech c) Video into frames
3	Build a feed forward neural network for prediction of logic gates.
4	Write a program for character recognition using: a) CNN b) RNN
5	Write a program to predict a caption for a sample image using : a) LSTM b) CNN
6	Write a program to develop : a) Autoencoders using MNIST Handwritten Digits. b) GAN for Generating MNIST Handwritten Digits.

INDEX

Sr. No.	Topic	Page Number
1	Lab Plan	
2	Lab Objective & Requirements	
3	List of Experiments	
4	Introduction to Lab	
5	Experiment No 1(a)	
6	Experiment No 1(b)	
7	Experiment No 2(a)	
8	Experiment No 2(b)	
9	Experiment No 2(c)	
10	Experiment No 3	
11	Experiment No 4(a)	
12	Experiment No 4(b)	
13	Experiment No 5(a)	
14	Experiment No 5(b)	
15	Experiment No 6(a)	
16	Experiment No 6(b)	

LAB PLAN

Total number of experiments 16

Total number of turns required 16

Number of turns required for

Experiment Number	Turns	Scheduled Day
Experiment 1		
Experiment 2		
Experiment 3		
Experiment 4		
Experiment 5		
Experiment 6		

Distribution of Lab Hours:

Attendance	05 minutes
Explanation of features of language	15 minutes
Explanation of experiment	30 minutes
Performance of experiment	40 minutes
File Checking	10 minutes
Viva / Quiz / Queries	20 minutes
Total	**120 minutes**

Lab Objective & Requirements

The Deep Learning Laboratory is designed to provide students with hands-on experience in applying deep learning techniques to real-world problems. The objectives are:

1. To understand the fundamentals of neural networks and deep learning architectures.
2. To implement algorithms such as feedforward networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformers.
3. To gain practical skills in data preprocessing, model training, optimization, and evaluation.
4. To explore applications of deep learning in domains such as computer vision, natural language processing (NLP), and speech recognition.
5. To provide exposure to popular deep learning frameworks such as TensorFlow and PyTorch.
6. To encourage innovation by developing and testing deep learning models for research and industrial applications.
7. To strengthen problem-solving skills by working on real-world datasets and projects.

Requirements for Deep Learning Lab

Hardware Requirements

- **High-Performance Computing Systems** with GPU support (NVIDIA CUDA-enabled GPUs, minimum 8–16 GB VRAM).
- Workstations with minimum configuration:
  - Processor: Intel i7/i9 or AMD Ryzen 7/9
  - RAM: 16–32 GB
  - Storage: 1 TB SSD (for datasets and model checkpoints)
- High-speed LAN/Wi-Fi connectivity.
- Dedicated server or cloud-based GPU instances (optional but recommended).

Software Requirements

- Operating System: Ubuntu/Linux (preferred) or Windows.
- Deep Learning Frameworks:
  - TensorFlow, Keras, PyTorch
  - Scikit-learn, OpenCV
- Development Tools:
  - Jupyter Notebook, Anaconda/Miniconda
  - VS Code / PyCharm IDE
- Libraries for Data Science:
  - NumPy, Pandas, Matplotlib, Seaborn
- Dataset Repositories: Kaggle, UCI, ImageNet (as required).

Deep learning Introduction

Deep learning is a subset of machine learning, which is a field of artificial intelligence (AI) that focuses on teaching machines to learn from data. Deep learning models are designed to mimic the way the human brain processes information and learns from it. They are capable of automatically learning representations of data through the use of neural networks with multiple layers. These neural networks are composed of interconnected nodes, or "neurons," that process and transform input data to produce an output. The term "deep" in deep learning refers to the multiple layers through which data is processed in these neural networks. Each layer of neurons extracts features from the input data, and these features are then passed on to the next layer for further processing. Deep learning models are known for their ability to automatically learn hierarchical representations of data, which can be used for tasks such as image and speech recognition, natural language processing, and many other applications. Deep learning has gained popularity and achieved remarkable success in recent years, particularly in areas such as computer vision and natural language processing, where it has outperformed traditional machine learning methods in many tasks. This success is largely attributed to the availability of large amounts of data, powerful computational resources (such as GPUs), and advances in neural network architectures and training algorithms.

Some popular deep learning frameworks and libraries include Tensor Flow, PyTorch, and Keras, which provide tools and APIs for building and training deep learning models. These frameworks have made it easier for researchers and developers to work with deep learning, leading to rapid advancements in the field.

Deep learning tasks

Deep learning, as a subset of machine learning, is particularly well-suited for tasks that involve complex data representations and large amounts of data. Some common deep learning tasks include:

Image Recognition and Classification:

Deep learning models are widely used for tasks such as image classification, where the goal is to categorize images into different classes (e.g., identifying objects in images).

Object Detection:

Object detection involves identifying and locating objects within images or videos. Deep learning models like convolution neural networks (CNNs) are commonly used for this task.

Semantic Segmentation:

Semantic segmentation involves labeling each pixel in an image with a class label, effectively creating a pixel-level understanding of the image. This task is often used in medical imaging and autonomous driving.

Speech Recognition:

Deep learning models, particularly recurrent neural networks (RNNs) and convolutional neural networks (CNNs) are used for speech recognition tasks, such as converting spoken language into text.

Natural Language Processing (NLP):

Deep learning has been highly successful in various NLP tasks, including language translation, sentiment analysis, text summarization, and named entity recognition.

Generative Models:

Generative models, such as Generative Adversarial Networks (GANs) and Variational Auto encoders (VAEs), are used to generate new data samples that are similar to a given training dataset. These models have applications in image generation, text-to-image synthesis, and more.

Recommender Systems:

Deep learning models can be used to build recommender systems that predict user preferences based on historical data, such as user-item interaction patterns in e-commerce or content consumption platforms.

Anomaly Detection:

Deep learning models can be used for anomaly detection in various domains, such as detecting fraudulent transactions in finance or identifying abnormal patterns in industrial equipment.

Time Series Forecasting:

Deep learning models, including recurrent neural networks (RNNs) and long short-term memory networks (LSTMs), are used for time series forecasting tasks, such as predicting stock prices or weather patterns.

These are just a few examples of the many tasks that deep learning can be applied to. Deep learning continues to advance rapidly, leading to improvements in performance across a wide range of applications.

Definition of Deep Learning: Deep learning is a subset of machine learning that focuses on modeling high-level abstractions in data through the use of complex architectures composed of multiple non-linear transformations. These architectures, often referred to as neural networks, are inspired by the structure and function of the human brain, with interconnected nodes, or "neurons," that process and transform data.

The "deep" in deep learning refers to the multiple layers through which data is processed in these neural networks. Each layer of neurons extracts features from the input data, and these features are then passed on to the next layer for further processing. Deep learning models are known for their ability to automatically learn hierarchical representations of data, which can be used for tasks such as image and speech recognition, natural language processing, and many other applications.

.

Deep Learning

Subject Code : 7CAI4-21

Lab Experiments

Exp No.-1(a)

Build a deep neural network model start with linear regression using Single variable

```
Import tensorflow as tf
Import numpy as np
Import matplotlib.pyplot as plt
# Generate some random data
np.random.seed (0)
X = np.random.rand (100, 1)
Y = 2 * X + 1 + np.random.rand (100, 1) * 0.1 # y = 2x + 1 + noise
# Define the model architecture
Model = tf.keras.Sequential ([tf.keras.layers.Dense (units=1, input shape= [1]) # Single input
neuron and single output neuron])
# Compile the model
model.compile (optimizer='sgd', loss='mean_squared_error')
# Train the model
History = model.fit(X, Y, epochs=100, verbose=0)
# Plot the training loss
Plt. Plot (history. history ['loss'])
plt.xlabel ('Epochs')
plt.ylabel ('Loss')
Plt. Show ()
# Make predictions
X_test = np. Array ([[0.2], [0.5], [0.8]])
Predictions = model. Predict (X_test)
Print (predictions)
```

1. What is Linear Regression Model?
Ans: Start by explaining the concept of linear regression, which is a simple model that predicts a continuous value (dependent variable) based on an input variable (independent variable) using a linear relationship.

2. What is Mathematical Formulation?
Ans: Describe the mathematical formulation of linear regression, such as the equation of a straight line (y = mx + c) where 'y' is the predicted value, 'm' is the slope, 'x' is the input variable, and 'c' is the intercept.

3. What is Neural Network Representation?
Ans: Introduce the idea of representing linear regression as a neural network with a single neuron. Explain that the neuron performs a weighted sum of the input (x) and adds a bias term to produce the output.

4. Explain the Activation Function
Ans: Mention that in the context of linear regression, the activation function is often just the identity function, as the output is a linear combination of the input.

5. What is Loss Function?
Explain that the loss function for linear regression is typically the mean squared error (MSE), which measures the difference between the predicted value and the actual value?

Exp No-2(b)

Build a deep neural network model start with linear regression using multiple variables

```
Import tensor flow as tf
Import numpy as np
Import matplotlib.pyplot as plt
# Generate some random data with two input features
np.random.seed (0)
X = np.random.rand (100, 2)
Y = 2 * X [: 0] + 3 * X [: 1] + 1 + np.random.rand (100) * 0.1 # y = 2x1 + 3x2 + 1 + noise
# Define the model architecture
Model = tf.keras.Sequential ([tf.keras.layers.Dense (units=1, input shape=[2])  # Two input
neurons and single output neuron])
# Compile the model
model.compile (optimizer='sgd', loss='mean_squared_error')
# Train the model
History = model.fit(X, Y, epochs=100, verbose=0)
# Plot the training loss
Plt. Plot (history. history ['loss'])
plt.xlabel ('Epochs')
plt.ylabel ('Loss')
Plt. Show ()
# Make predictions
X_test = np.array ([[0.2, 0.3], [0.5, 0.6], [0.8, 0.9]])
Predictions = model. Predict (X_test)
Print (predictions)
```

Viva question

1. **Describe a well-known law or natural phenomena that you could model with linear regression.**

Ans: Many common laws have proportional relations and can be described by linear relationships. These can therefore be modeled with linear regression. A few famous examples are Ohm's law-related by V=IR or Newton's second law given by F=ma. On close observation, it is apparent that each of these laws has the form y=mx+c where c=0, or are essentially described by lines that pass through the origin.

2. Is it necessary to visualize the data when you have fitted a line? Why or why not?

Ans: It is crucial to visualize the data when you fit a line because numerically fitting a line is easy with pure numerical analysis or methods like Least Squares regression. But determining whether these fitted lines make any sense requires further analysis of which visualizing the data is one of the easiest ways.

3. Does correlation imply causation? Why or why not?

No, while correlation is popularly used to provide information on the extent and direction of the linear relationship between two variables and can be used to determine whether a variable can be used to predict another, a high correlation does not imply causation.

4. Is linear regression suitable for time series analysis?

While linear regression can be used for time series analysis and generally yield workable results, the performance is not particularly remarkable. The two main factors for this are: Time series generally have seasonal or periodic trends (such as peak seasons or even peak hours), which might be treated as outliers in linear regression and hence not appropriately accounted for.

5. Explain with an example when you would and would not use this.

Extrapolation is essentially predicting values of the target function for parameter values outside the range of those observed during training. While extrapolation could reasonably work well in some cases, such as predicting the voltage in Ohm's law, it can also result in meaningless results. One easy example to explain this can be to extrapolate the decreasing rainfall trend at the end of the rainy season. If the extrapolation is done unchecked, the model could predict negative rain after a period that is about as nonsensical as it gets.

Exp No-2(a,b,c)

Write a program to convert: a) Speech into text b) Text into speech c) Video into frames

a)

```
Import speech recognition as sr
  # Initialize the recognizer
Recognizer = sr. Recognizer ()
# Record audio from the microphone
With sr. Microphone () as source:
   Print ("Say something...")
   Audio = recognizer. Listen (source)
# recognizes speech using Google Web Speech API Try:
   Text = recognizer.recognize_google (audio)
   Print ("You said:” text)
Except sr.UnknownValueError:
   Print ("Could not understand audio")
Except sr.RequestError as e:
   Print ("Could not request results; {0}".format (e))
```

b)

```
Import pyttsx3
# Initialize the text-to-speech engine
Engine = pyttsx3.init ()
# Convert text to speech
Text = "Hello, how are you?"
Engine. Say (text)
engine.runAndWait ()
```

c)

```
Import cv2
# Open the video file
Video path = "path/to/your/video.mp4"
Cap = cv2.VideoCapture (video path)
# Read and save frames
Frame count = 0
While True:
   Ret, frame = cap. Read ()
   If not ret:
      Break
   Frame count += 1
   Frame path = f"frame_ {frame count}.jpg"
   cv2.imwrite (frame path, frame)
# Release the video capture object
Cap. Release ()
```

Viva question

1. What is Naive Bayes algorithm, when we can use this algorithm in NLP?

Ans: Naive Bayes algorithm is a collection of classifiers which works on the principles of the Bayes' theorem. This series of NLP model forms a family of algorithms that can be used for a wide range of classification tasks including sentiment prediction, filtering of spam, classifying documents and more.

2. Explain Dependency Parsing in NLP?

Ans: Dependency Parsing, also known as Syntactic parsing in NLP is a process of assigning syntactic structure to a sentence and identifying its dependency parses. This process is crucial to understand the correlations between the "head" words in the syntactic structure. The process of dependency parsing can be a little complex considering how any sentence can have more than one dependency parses. Multiple parse trees are known as ambiguities. Dependency parsing needs to resolve these ambiguities in order to effectively assign a syntactic structure to a sentence.

3. What is text Summarization?

Ans: Text summarization is the process of shortening a long piece of text with its meaning and effect intact. Text summarization intends to create a summary of any given piece of text and outlines the main points of the document. This technique has improved in recent times and is capable of summarizing volumes of text successfully.

4. What is NLTK? How is it different from Spacy?

Ans: NLTK or Natural Language Toolkit is a series of libraries and programs that are used for symbolic and statistical natural language processing. This toolkit contains some of the most powerful libraries that can work on different ML techniques to break down and understand human language. NLTK is used for Lemmatization, Punctuation, Character count, Tokenization, and Stemming.

5. What is information extraction?

Ans: Information extraction in the context of Natural Language Processing refers to the technique of extracting structured information automatically from unstructured sources to ascribe meaning to it. This can include extracting information regarding attributes of entities, relationship between different entities and more. The various models of information extraction include:

- Tagger Module
- Relation Extraction Module
- Fact Extraction Module

Exp No-3

Build a feed forward neural network for prediction of logic gates.

```
Import tensor flow as tf
Import numpy as np
# Define the XOR gate inputs and outputs
X = np.array ([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=np.float32)
Y = np.array ([[0], [1], [1], [0]], dtype=np.float32)
# Define the neural network architecture
Model = tf.keras.Sequential ([
    tf.keras.layers.Dense (units=2, input shape= (2,), activation='sigmoid'),
    tf.keras.layers.Dense (units=1, activation='sigmoid')])
# Compile the model
model.compile (optimizer='adam', loss='binary_crossentropy', metrics= ['accuracy'])
# Train the model
History = model.fit(X, Y, epochs=1000, verbose=0)
# Evaluate the model
Loss, accuracy = model. Evaluate(X, Y)
Print (f"Loss: {loss}, Accuracy: {accuracy}")
# Make predictions
Predictions = model. Predict(X)
Print ("Predictions :")
For me in range (Len(X)):
    Print (f"Input: {X[i]}, Predicted Output: {predictions[i]}, Actual Output: {Y[i]}")
```

Viva question

1. What are logic gates?

Ans. Logic gates are the fundamental building blocks of digital systems. They are circuits in which the output depends upon the inputs according to some logic rules.

2. What is a positive logic system?

Ans. A positive logic system is one in which the higher of the two voltage levels represents logic 1 and the lower of the two voltage levels represents logic 0.

3. What is a negative logic system?
Ans. A negative logic system is one in which the higher of the two voltage levels represents logic 0 and the lower of the two voltage levels represent logic 1.

4. What is an AND gate?

Ans. An AND gate is a logic gate with two or more inputs and only one output. It outputs a 1 only when each one of its in is 1. It outputs a 0 even if one of its inputs is a 0. It is a basic gate. It is also called an all or nothing gate. It performs AND operation.

5. What is an OR-gate?

Ans. An OR gate is a logic gate with two or more inputs and only one output. It outputs a 1 even if one of its inputs is a 1. It outputs a 0 only when each one of its input is a 0. It is also called an any or all gate. It is a basic gate. It performs OR operation.

Exp no-4(a)

Write a program for character recognition using: CNN

```
Import tensor flow as tf
From tensorflow.keras.datasets import mnist
From tensorflow.keras.models import Sequential
From tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
From tensorflow.keras.utils import to categorical
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data ()
# Preprocess the data
x_train = x_train. Reshape ((x_train. Shape [0], 28, 28, 1)).as type ('float32') / 255
x_test = x_test. Reshape ((x_test. Shape [0], 28, 28, 1)).as type ('float32') / 255
y_train = to categorical (y_train)
y_test = to categorical (y_test)
# Define the CNN model
model = Sequential ([
   Conv2D (32, (3, 3), activation='relu', input shape= (28, 28, 1)),
   MaxPooling2D ((2, 2)),
   Conv2D (64, (3, 3), activation='relu'),
   MaxPooling2D ((2, 2)),
   Flatten (),
   Dense (64, activation='relu'),
   Dense (10, activation='surtax')]
# Compile the model
model.compile (optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
# Train the model
History = model.fit (x_train, y_train, epochs=5, batch size=64, validation data= (x_test, y_test))
# Evaluate the model
Loss, accuracy = model. Evaluate (x_test, y_test)
print (fattest accuracy: {accuracy}")
```

Viva question

1. What is a Convolution Neural Network?
Ans: A Convolution Neural Network is a type of neural network that is designed to work with images. It is made up of a series of layers, each of which is responsible for detecting certain features in an image. The first layer might detect edges, for example, while the second layer might detect shapes, and so on.

2. Can you explain the architecture of a convolution neural network (CNN)?
Ans: - A convolution neural network is a type of neural network that is typically used for image recognition tasks. The architecture of a CNN typically consists of an input layer, a series of convolution layers, a pooling layer, and an output layer. The convolution layers are responsible for extracting features from the input image, while the pooling layer is responsible for reducing the dimensionality of the feature map. The output layer is responsible for classification.

3. How does a CNN differ from a fully connected neural network?
Ans: A CNN differs from a fully connected neural network in a few key ways. First, CNNs are designed to take advantage of the spatial structure of data, meaning that they are better able to identify patterns in images, for example. Additionally, CNNs use smaller filters and pooling layers in order to reduce the dimensionality of the data and make the network more efficient. Finally, CNNs are often used in conjunction with other machine learning algorithms, such as support vector machines, in order to improve performance.

4. Can you name some popular applications where CNNs are used extensively?
Ans: CNNs are used extensively in computer vision applications such as image classification, object detection, and face recognition.

5. What do you understand by feature extraction and how is it performed?
Ans: Feature extraction is a process of reducing the amount of data in an image while still retaining the important information. This is done by identifying and extracting the features that are most important for the task at hand. For example, when extracting features for a facial recognition task, you would want to focus on extracting features that are most relevant to distinguishing one face from another, such as the shape of the nose or the placement of the eyes.

Exp no-4(b)

Write a program for character recognition using: RNN

```
Import tensor flow as tf
From tensorflow.keras.datasets import mnist
From tensorflow.keras.models import Sequential
From tensorflow.keras.layers import SimpleRNN, Dense
From tensorflow.keras.utils import to_categorical
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data ()
# Preprocess the data
x_train = x_train.reshape ((x_train. Shape [0], 28, 28)).as type ('float32') / 255
x_test = x_test. Reshape ((x_test. Shape [0], 28, 28)).as type ('float32') / 255
y_train = to_categorical (y_train)
y_test = to_categorical (y_test)
# Define the RNN model
Model = Sequential ([
   Simpler (128, input shape= (28, 28)),
   Dense (10, activation='surtax')])
# Compile the model
model.compile (optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
# Train the model
History = model.fit (x_train, y_train, epochs=5, batch size=64, validation data= (x_test, y_test))
# Evaluate the model
Loss, accuracy = model. Evaluate (x_test, y_test)
Print (fattest accuracy: {accuracy}")
```

Viva question

1. What Is An RNN?

Ans: An RNN (Recurrent Neural Network) is a neural network that processes sequential data using recurrent connections. Specifically, it suits tasks involving sequences, such as natural language processing, speech recognition, and time series analysis. RNNs have an internal memory that allows them to retain information from previous inputs and use it to make predictions or decisions based on the context of the entire sequence.

2. How Does An RNN Differ From Other Neural Networks?

Ans: The critical difference between RNNs and other neural networks is their ability to handle sequential data. RNNs maintain hidden states that carry information from previous time steps. This recurrent nature enables RNNs to model temporal dependencies and capture the sequential patterns inherent in the data. In contrast, tasks where input order is unimportant better suit feed forward networks.

3. How Do RNNs Handle Variable-Length Inputs?

Ans: RNNs handle variable-length inputs by processing the data sequentially, a one-time step at a time. They iterate through the input sequence, maintaining hidden states that carry information from previous time steps. This allows RNNs to handle inputs of different sizes and capture dependencies across the entire series.

4. What Is The Architecture Of An RNN?

The architecture of an RNN consists of recurrent connections that enable information to be passed from one-time step to the next. At each time step, the RNN takes an input, combines it with the previous hidden state, and produces an output and a new hidden state. The hidden state serves as the memory of the network and retains information from past inputs. This architecture allows RNNs to process sequences of arbitrary length while considering the contextual information from previous inputs.

5. What Is A Sequence-To-Sequence RNN?

A sequence-to-sequence RNN is an RNN model that takes a sequence as input and produces another as output. Using them in tasks such as machine translation, where the input sequence (source language) is translated into an output sequence (target language). Sequence-to-sequence RNNs consist of an encoder that processes the input sequence and a decoder that generates the output sequence based on the encoded information.

Exp No-5(a)

Write a program to predict a caption for a sample image using LSTM

```
Import tensor flow as tf
From tensorflow.keras.applications import VGG16
From tensorflow.keras.models import Model
From tensorflow.keras.layers import Input, LSTM, Embedding, Dense, Attention, Concatenate, Time Distributed
From tensorflow.keras.preprocessing.sequence import pad sequences
From tensorflow.keras.preprocessing.text import Tokenizer
From tensorflow.keras.utils import to_categorical
From tensorflow.keras.callbacks import Model Checkpoint
Import numpy as np
Import pickle
Import os
From PIL import Image

# Path to the MS-COCO dataset
data_dir = '/path/tm/mscoco'
train_image_dir = os.path.join (data_dir, 'train2014')
val_image_dir = os.path.join (data_dir, 'val2014')
annotations_dir = os.path.join (data_dir, 'annotations')

# Load the training captions
With open (os.path.join (annotations_dir, 'captions_train2014.json'), 'rib') as f: train captions = json.load (f)
# preprocess captions
Captions = []
For caption in train captions ['annotations']:
   Captions. Append (caption ['caption'])

# tokenize captions
Tokenize = Tokenize ()
tokenizer.fit_on_texts (captions)
vocab_size = len (tokenizer.word_index) + 1

# save tokenize
With open ('tokenizer.pkl', 'wb') as f:
   Pickle. Dump (tokenizer, f)

# generate sequences from tokens
Sequences = tokenizer.texts_to_sequences(captions)
max_sequence_length = max ([len (seq) for seq in sequences])
Padded sequences = pad sequences (sequences, maxlen=max_sequence_length, padding='post')
```

```
# Load pre-trained VGG16 model
image_model = VGG16 (weights='image net', include top=False)
new_input = image_model. Input
Hidden layer = image_model. Layers [-1].output
image_features_extract_model = Model (inputs=new_input, outputs=hidden layer)

# Define the LSTM model for image captioning
Input image = Input (shape= (224, 224, 3))
Features = image_features_extract_model (input image)
Features = Attention () ([features, features])
decoder1 = LSTM (256) (features)
output1 = Dense (vocab_size, activation='surtax')(decoder1)
Model = Model (inputs=input image, outputs=output1)
model.compile (loss='categorical_crossentropy', optimizer='adam')
# Train the model
# (Note: This is a simplified example. In practice, you would use the full dataset and implement more sophisticated training techniques.)
model.fit (x_train, y_train, epochs=10, batch size=32)
# save the model
Model. Save ('image_captioning_model.h5')
# Load the saved tokenized
With open ('tokenizer.pkl', 'rb') as f:
   tokenized = pickle. Load (f)
# Load the saved model
Model = tf.keras.models.load_model ('image_captioning_model.h5')
# Load an example image
example_image_path = '/path/tm/your/example/image.jpg'
img = Image. Open (example_image_path)
Img = img. Resize ((224, 224))
img = np.array (img) / 255.0
Img = np.expand_dims (img, axis=0)
# Generate a caption for the example image
Features = image_features_extract_model. Predict (img)
Decoder input = np.zeros ((1, 1, 256))
decoder input[0, 0, :] = features[0, 0, :]
Caption = []
For i in range (max_sequence_length):
   Output = model. Predict ([decoder input, img])
   Word index = np.argmax (output)
   Word = tokenizer.index_word [word index]
   Caption. Append (word)
   If words == '<end>':
      Break
   decoder input[0, 0, :] = features[0, 0, :]
Caption text = ' '.join (caption [:-1])
# Exclude the <end> token
Print ("Predicted caption:” caption text
```

Viva question

1. **What is an LSTM layer, and why is it used in deep learning?**

Answer: An LSTM layer is a type of recurrent neural network (RNN) layer designed to capture long-term dependencies in sequential data. It is used in deep learning to model and make predictions on time series data, natural language processing tasks, and other sequential data applications.

2. **Explain the vanishing gradient problem. How does LSTM address this issue compared to traditional RNNs?**

Answer: The vanishing gradient problem occurs when gradients become too small during back propagation, making it challenging to update the weights of deep neural networks. LSTMs address this by using a gating mechanism that allows them to learn when to update and forget information, mitigating the vanishing gradient problem better than traditional RNNs.

3. **What are the key components of an LSTM cell?**

Answer: An LSTM cell consists of three key components: the input gate, the forget gate, and the output gate. These gates control the flow of information into and out of the cell, and the cell state stores the information over time.

4. **How does the input gate work in an LSTM cell**

Answer: The input gate controls what information to add to the cell state. It uses a sigmoid activation function to determine which values should be updated and a tanh activation function to create a candidate update.

5. **Explain the role of the forget gate in an LSTM cell?**

Answer: The forget gate decides which information from the previous cell state should be retained and which should be discarded. It uses a sigmoid activation function to produce values between 0 and 1 for each component of the cell state.

Exp no-5(b)

Write a program to predict a caption for a sample image using CNN

```
Import torch
Import torchvision.transforms as transforms
From PIL import Image
Import json
# Load pre-trained CNN-RNN model
Model path = 'path_to_your_trained_model.pth'
With open ('vocab.json', 'r') as f:
   Vocab = json. Load (f)
# define transformations for the input image
Transform = transforms. Compose ([
   Transforms. Resize ((224, 224)),
   transforms.ToTensor (),
   Transforms. Normalize (mean= [0.485, 0.456, 0.406], STD= [0.229, 0.224, 0.225]),])
# Load the image
Image path = 'path_to_your_image.jpg'
Image = Image. Open (image path).convert ('RGB')

# apply transformations
Image = transform (image).unsqueeze (0)
# Load the model
Model = torch. Load (model path)
model.eval ()

# generate caption
With torch.no_grad ():
   Features = model. Encoder (image)
   sampled_ids = model.decoder.sample (features)
   sampled_ids = sampled_ids [0].cpu ().numpy ()

# Convert indices to words
Predicted caption = []
For word_id in sampled_ids:
   Word = vocal [STR (word_id)]
   Predicted caption. Append (word)
   If words == '<end>':
      Break

Predicted caption = ' '.join (predicted caption [:-1])
# Remove <end> token
Print ('Predicted caption:’ predicted caption)
```

Viva question

1. What are activation functions? Draw the graphs of commonly used activation functions.

Activation functions are the functions that are used to evaluate the output of a node in a neural network. It is the sum of two quantities- the weighted sum of the input values and a bias term. The output value decides whether a particular neuron in a specific neural network layer is activated.

2. What are the different types of pooling layers in CNN architecture?

Assuming the filter of size 2x2, the three common ways of creating a pooling layer in a neural network is:
Max Pooling: It returns the maximum value out of the 4 elements in the receptive field.

Average Pooling: In this case, the average of the four values is computed to produce the output.

L_2 Pooling: The square root of the sum of the squares of the four elements is computed.

3. What is padding? Why do we need padding?

Padding refers to the process of adding a layer of zeros/ones to the sides of a matrix. We perform padding so that we can smoothly implement the convolution operation.

4. What is transfer learning?

Transfer learning is the process of utilizing the knowledge gained by the pre-trained model and its knowledge to solve a more complex problem.

5. What is a flattening layer in CNN architecture?

The flattening layer is usually towards the end of the CNN architecture, and it is used to transform all the two-dimensional matrices into a single lengthy vector. The output of this layer is passed to the fully-connected layer. Where a = (m-1)/2; m denotes the size of w.

6. List a few applications of a CNN.

A few applications of CNN are:

Face Detection and Recognition

Image Classification

Image Segmentation

Edge detection

Document Analysis

Image Colorization

Recommendation System

Exp No-6(a)

Write a program to develop: Auto encoders using MNIST Handwritten Digits.

```
Import torch
Import torch.nn as nn
Import torchvision
Import torchvision.transforms as transforms
Import matplotlib.pyplot as plt
# hyper parameters
num_epochs = 10
Batch size = 128
Learning rate = 0.001

# MNIST dataset
Transform = transforms. Compose ([
   transforms.ToTensor (),
   Transforms. Normalize ((0.5,), (0.5,))])

Train dataset = torchvision.datasets.MNIST (root='./data', train=True, transform=transform, download=True) train loader = torch.utils.data.DataLoader(dataset=train dataset, batch size=batch size, shuffle=True)

# Auto encoder model
Class Auto encoder (nn.Module):
   def __init__ (self):
      Super (Auto encoder, self).__init__ ()
      Self. Encoder = nn.Sequential (
         Nonlinear (28*28, 128),
         nn.ReLU (),
         Nonlinear (128, 64),
         nn.ReLU (),
         Nonlinear (64, 32),
         nn.ReLU ()
      )
      Self. Decoder = nn.Sequential (
         Nonlinear (32, 64),
         nn.ReLU (),
         Nonlinear (64, 128),
         nn.ReLU (),
         Nonlinear (128, 28*28),
         nn.Tanh ()
      )

   Def forward (self, x):
      x = x.view (q.size (0), -1)
      Encoded = self. Encoder(x)
      Decoded = self. Decoder (encoded)
```

```
        Decoded = decoded. View (-1, 1, 28, 28)
        Return decoded

Model = Auto encoder ()

# Loss and optimizer
Criterion = nn.MSELoss ()
Optimizer = torch.optim.Adam (model. Parameters (), lr=learning rate)

# Training the auto encoder
Total step = len (train loader)
For epoch in range (num_epochs):
    For i, (images, _) in enumerate (train loader):
        # Forward pass
        Outputs = model (images)
        Loss = criterion (outputs, images)

        # backward and optimize
        optimizer.zero_grad ()
        Loss. Backward ()
        optimizer. Step ()

        If (i+1) % 100 == 0:
            Print ('Epoch [{}/ {}], Step [{}/ {}], Loss: {:.4f}'
                .format (epoch+1, num_epochs, i+1, total step, loss. Item ()))

# Test the auto encoder
With torch.no_grad ():
    For images, _ in train loader:
        Reconstructed images = model (images)
        Break

# Display original and reconstructed images
Fig, axes = plt. Subplots (rows=2, cols=10, share=True, share=True, fig size=(20, 4))
For images, row in zip ([images, reconstructed images], axes):
    For img, ax in zip (images, row):
        ax.imshow (img. Squeeze (), cmap='gray')
        ax.get_xaxis ().set visible (False)
        ax.get_yaxis ().set visible (False)

Plt. Show ()
```

Viva question

Q1. **What is the difference between AI, Machine Learning, and Deep Learning?**

Ans. **AI (Artificial Intelligence):** Artificial Intelligence refers to the study, development, and application of computer techniques that allow computers to acquire certain skills of human intelligence.
ML (Machine Learning): Machine learning is a subset of Artificial Intelligence where people "train" machines to recognize patterns based on data and make their predictions. ML algorithms are mathematical algorithms that allow machines to learn by imitating the way we humans learn.
DL (Deep Learning): Deep Learning is a subset of ML in which the machine is able to reason and draw its own conclusions, learning by itself. Deep Learning uses algorithms that mimic human perception inspired by our brain and the connection between neurons. Most deep learning methods use neural network architecture.

Q2. What is a neural network?

Ans. A neural network is a system of programs and data structures that approximates the functioning of the human brain. A neural network usually involves a large number of processors operating in parallel, each having its own small sphere of knowledge and access to data in its local memory.
A neural network is initially "trained" or fed with large amounts of data and rules about relationships (e.g. "a grandparent is older than a person's father"). A program can then tell the network how to behave in response to an external stimulus (e.g. input from a computer user interacting with the network) or it can initiate the activity itself, within limits of their access to the external world.
Deep learning uses neural networks to learn useful representations of features directly from data. For example, you can use a pre-trained neural network to identify and remove artifacts such as noise from images.

Q3. What is the idea behind the GANs?
Ans. The Generative Adversarial Network (GAN) is a very popular candidate in the field of machine learning that has showcased its potential to create realistic-looking images and videos. GANs consist of two networks (D & G) where –
D ="discriminating" network
G = "Generative" network.
The goal is to create data: images, for example, that cannot be distinguished from actual images. Suppose we want to create an adversarial example of a cat. Network G will generate images. Network D will classify the images according to whether it is a cat or not. The cost function of G will be constructed in such a way that it tries to "trick" D into always classifying its output as a cat.

Q4. In the neural network, if the hidden layer has a sufficient number of units, can it approximate any continuous function?

Ans. The Universal Approximation Theorem states that a simple feed forward neural network (i.e. multilayer perceptron) with a single hidden layer and a standard activation function can approximate any continuous function if the hidden layer has a sufficient number of units. If the function jumps around or has large gaps, it cannot be approximated.

Q5. **How will you solve the gradient explosion problem?**

Ans. There are many ways to solve the gradient explosion problem. Some of the best experimental methods are –

Redesign the network model – In deep neural networks, the gradient explosion can be solved by redesigning the network with fewer layers. Using a smaller batch size is also good for network training. In recurrent neural networks, updating in fewer previous time steps during training (truncated back propagation over time) can alleviate the gradient burst problem.

Use the ReLU trigger function – In deep multilayer perceptron neural networks, gradient explosion can occur due to activation functions, such as the previously popular Sigmoid and Tanh functions. Using the ReLU trigger function can reduce gradient bursts. Adopting the ReLU trigger function is one of the most popular practices for hidden layers.

Exp no-6(b)

Write a program to develop: GAN for Generating MNIST Handwritten Digits.

```
Import torch
Import torch.nn as nn
Import torch.optim as optim
Import torch vision
Import torchvision.transforms as transforms
Import matplotlib.pyplot as plt
# Hyper parameters
num_epochs = 50
Batch size = 128
learning rate = 0.0002
latent size = 100
# MNIST dataset
Transform = transforms. Compose ([transforms.ToTensor (), Transforms. Normalize ((0.5,), (0.5,))])
Train dataset = torchvision.datasets.MNIST (root='./data', train=True, transform=transform, download=True) train loader = torch.utils.data.DataLoader(dataset=train dataset, batch size=batch size, shuffle=True)

# Generator model
Class Generator (nn.Module):
   Def __init__ (self):
      Super (Generator, self).__init__ ()
      Self. Model = nn.Sequential (
         nonlinear (latent size, 256),
         nn.ReLU(),
         nonlinear (256, 512),
         nn.ReLU(),
         nonlinear(512, 1024),
         nn.ReLU (),
         Nonlinear (1024, 28*28),
         nn.Tanh ()
      )
   Def forward (self, x):
      Return self. Model(x)
# Discriminator model
Class Discriminator (nn.Module):
   Def __init__ (self):
      Super (Discriminator, self).__init__()
      Self. Model = nn.Sequential (
         Nonlinear (28*28, 1024),
         nn.ReLU (),
         nn.Dropout(0.3),
         nn.Linear(1024, 512),
         nn.ReLU(),
         nn.Dropout (0.3),
```

```
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = x.view (x.size(0), -1)
        Return self.model(x)

# Create the models
Generator = Generator ()
Discriminator = Discriminator ()

# Loss function and optimizer
Criterion = nn.BCELoss ()
optimizer_g = optim.Adam (generator.parameters (), lr=learning_rate)
optimizer_d = optim.Adam (discriminator.parameters (), lr=learning_rate)

# Training the GAN
For epoch in range (num_epochs):
    For i, (real_images, _) in enumerate (train_loader):
        batch_size = real_images.size (0)
        real_images = real_images.view (batch_size, -1)

        # Train discriminator with real images
        optimizer_d.zero_grad ()
        real_labels = torch.ones (batch_size, 1)
        real_pred = discriminator (real_images)
        real_loss = criterion (real_pred, real_labels)
        real_loss.backward ()

        # Train discriminator with fake images
        Noise = torch.randn (batch_size, latent_size)
        fake_images = generator (noise)
        fake_labels = torch.zeros (batch_size, 1)
        fake_pred = discriminator (fake_images.detach ())
        fake_loss = criterion (fake_pred, fake_labels)
        fake_loss.backward ()
        optimizer_d.step ()

        # Train generator
        optimizer_g.zero_grad ()
        Noise = torch.randn (batch_size, latent_size)
        generated_images = generator (noise)
        Labels = torch.ones (batch_size, 1)
```

```
        Pred = discriminator (generated_images)
        loss_g = criterion (pred, labels)
        loss_g.backward ()
        optimizer_g.step ()

        If (i+1) % 200 == 0:
           Print (f'Epoch [{epoch+1}/ {num_epochs}], Step [{i+1}/ {len (train_loader)}],'
                F'D_loss: {real_loss.item () +fake_loss.item ():.4f}, G_loss: {loss_g.item ():.4f}')

# generate and display some fake images
with torch.no_grad():
    Noise = torch.randn (10, latent_size)
    generated_images = generator (noise).view (-1, 28, 28)

fig, axes = plt.subplots (1, 10, figsize=(20, 2))
For ax, img in zip (axes, generated_images):
    ax.imshow (img.cpu (), cmap='gray')
    ax.axis ('off')

plt.show ()
```

Viva question

1. What are GANs

Ans: - Generative adversarial networks, also known as **GANs** are deep generative models and like most generative models they use a differential function represented by a neural network known as a Generator network. GANs also consist of another neural network called Discriminator network.

2. Where are GANs used

Ans: Most of the applications for GANs have been in the field of computer vision. More specifically GANs are good at generating images that have never been seen before. Some of the models that have used GANs are described below:

StackGAN model: StackGAN model is good at generating images from the description. Text to image synthesis is one of the use cases for Generative Adversarial Networks (GANs) that has many industrial applications. Synthesizing images from text descriptions is a very hard task, as it is very difficult to build a model that can generate images that reflect the meaning of the text.

3. What is handwritten digit generation using GAN?

Ans: Handwritten-digit-generation-using-DCGAN A Generative Adversarial Network (GAN) is a class of machine learning systems invented by Ian Good fellow in 2014. Two neural networks contest with each other in a game (in the sense of game theory, often but not always in the form of a zero-sum game).

4. How does GAN generate data?

Ans: Generative adversarial networks, among the most important machine learning breakthroughs of recent times, allow you to generate useful data from random noise. Instead of training one neural network with millions of data points, you let two neural networks contest with each other to figure things out.

5. What is the principle of GAN?

Ans: A generative adversarial network (GAN) has two parts: The generator learns to generate plausible data. The generated instances become negative training examples for the discriminator. The discriminator learns to distinguish the generator's fake data from real data.